

Documentação - Fluxo de Rotas JP-Cashflow

■ Fluxo de Navegação - JP Cashflow (Middleware → Página Inicial)

1. Middleware do App (`middleware.ts`)

Objetivo: Identificar o subdomínio e redirecionar corretamente.

Etapas:

1. Captura o host via `request.headers.get("host")`.
2. Extrai o subdomínio (`jp` de `jp.localhost:3000``).
3. Se o path for ``/``, redireciona para ``/inicio``.
4. Se for ``/login`` e tiver ``token``, redireciona para ``/dashboard``.
5. Para outras rotas com subdomínio, faz ``rewrite`` da URL.
6. Se não houver subdomínio, segue com ``NextResponse.next()``.

2. Página Inicial (`/inicio`)

- Verifica se o subdomínio é válido via API pública.
- Se token for válido e corresponde ao subdomínio, redireciona para ``/dashboard``.
- Se o token for de outro usuário, redireciona para ``/login?user=subdominio``.
- Se não houver token, também redireciona para ``/login?user=subdominio``.

3. Página de Login (`/login`)

- Lê o token e o parâmetro ``user`` da URL.
- Se já estiver logado:
 - Se ``?user`` for o mesmo: vai para ``/dashboard``
 - Se ``?user`` for diferente: força logout com ``<ForceLogout />``
- Se não estiver logado: mostra o formulário de login.

4. Componente ``<ForceLogout />``

- Limpa o cookie ``token``.
- Reseta o contexto global.
- Redireciona para ``/login?user=subdominio``.

5. Página do Dashboard (`/dashboard`)

- Server Component:
 - Valida o token.
 - Se inválido ou ausente, redireciona para ``/login``.
 - Se válido, renderiza ``<DashboardClient />``.
- Client Component (``DashboardClient``):
 - Renderiza gráficos e componentes usando ``DashboardProvider``.
 - Cada gráfico chama uma função ``fetch`` com o token via ``Authorization``.

■ Fluxo da API (Middleware + Categorias de Rotas)

■ Middleware da API (`src/middleware.ts`)

Objetivo: Controlar o acesso às rotas com base na origem e token JWT.

Tipos de Origem Permitidas:

- `http://localhost:3000`
- `https://jp-cashflow.app`
- `http://*.localhost:3000`
- `https://*.jp-cashflow.app`

Classes de Rotas:

- Public Global (/api/public/global) → Sem token, qualquer origem.
- Public Restrita (/api/public/restrita) → Sem token, origem permitida.
- Private Global (/api/private/global) → Token obrigatório, origem livre.
- Private Restrita (/api/private/restrita) → Token obrigatório, origem permitida.

Etapas no Middleware:

1. Se método for OPTIONS, responde com headers CORS.
2. Se rota for privada restrita:
 - Bloqueia se a origem não for permitida.
 - Valida token com validateToken.
3. Se rota for privada global:
 - Permite qualquer origem.
 - Valida token com validateToken.
4. Se rota for pública restrita:
 - Bloqueia origem não permitida.
5. Se rota for pública global:
 - Permite qualquer origem.
6. Para demais rotas:
 - Libera se origem for válida.

Log completo por requisição:

- NOVA REQUISIÇÃO
- PATHNAME: /api/private/restrita/graficos/estatisticaDespesas
- ORIGIN: http://jp.localhost:3000
- TOKEN: Token recebido
- isInternalRequest: false
- isLocalhostSubdomain: true
- isProductionSubdomain: false
- isAllowedOrigin: false
- isAllowedOriginFinal: true

■ Endpoints Protegidos (/api/private/restrita/graficos/...)

Cada endpoint:

- Extrai token do Authorization.
- Valida via verifyToken.
- Extrai parâmetros da URL (periodId, etc).
- Executa queries via prisma.\$queryRaw.
- Retorna JSON com os dados.

- Tudo isso garante que apenas usuários autenticados e em ambientes válidos possam acessar seus dados.